

Examen Práctico - Sistemas Distribuidos

Instrucciones Generales

1. **Formato:** Trabajo individual
2. **Entrega:** Repositorio GitHub con código y documentación
3. **Tecnologías:** Python, Docker, MongoDB

Objetivo

Implementar un sistema distribuido que aplique los conceptos vistos en clase: **procesos**, **hilos**, **escalabilidad** y **comunicación distribuida**.

Parte 1: Sistema Básico de Procesamiento Distribuido

Descripción:

Crear un sistema que procese múltiples tareas usando tanto **procesos** como **hilos**, comparando su rendimiento.

Requisitos:

```
python
# Esqueleto base proporcionado
import multiprocessing
import threading
import time
import random

class TaskProcessor:
    def __init__(self):
        self.tasks_completed = multiprocessing.Value('i', 0)

    def process_task(self, task_id, difficulty):
        """Simula procesamiento de 4 tareas con diferente dificultad"""
```

```
# TODO: Implementar procesamiento
pass

def run_with_threads(self, tasks):
    """Ejecutar tareas usando hilos"""
    # TODO: Implementar
    pass

def run_with_processes(self, tasks):
    """Ejecutar tareas usando procesos"""
    # TODO: Implementar
    pass
```

Tareas específicas:

1. **Completar la implementación** del procesador de tareas
2. **Generar 4 tareas** con dificultad aleatoria (1-5)
3. **Medir tiempos** de ejecución para ambos enfoques
4. **Analizar** cuándo es mejor usar procesos vs hilos

Parte 2: Sistema de Almacenamiento Distribuido

Descripción:

Implementar un sistema simple de almacenamiento distribuido usando **MongoDB en Docker** con 2 nodos.

Requisitos:

```
yaml
# docker-compose.yml
version: '3.8'
services:
  mongo1:
    image: mongo:latest
    ports: ["27017:27017"]

  mongo2:
    image: mongo:latest
    ports: ["27018:27017"]

  app:
    build: .
    depends_on: [mongo1, mongo2]
```

Tareas específicas:

1. **Configurar** dos instancias de MongoDB
2. **Implementar** clase `DistributedStorage` que:
 - Conecte a ambas bases de datos
 - Distribuya documentos entre nodos (sharding básico)
 - Permita búsqueda en ambos nodos
3. **Probar** con 10 documentos de ejemplo

Parte 3: Balanceador de Carga Simple

Descripción:

Crear un balanceador de carga básico que distribuya peticiones entre dos servidores web.

Requisitos:

python

```
class LoadBalancer:
    def __init__(self, servers):
        self.servers = servers
        self.current_index = 0

    def round_robin(self):
        # TODO: Implementar algoritmo round-robin
        pass

    def least_connections(self):
        # TODO: Implementar algoritmo least connections
        pass
```

Tareas específicas:

1. **Implementar** dos algoritmos de balanceo
2. **Simular** 50 peticiones concurrentes
3. **Comparar** distribución de carga entre algoritmos
4. **Análisis** del desempeño del sistema

Entregables Requeridos

Repositorio GitHub debe contener:

```
└─ APELLIDO_NOMBRE/
  │
  ├─ README.md (explicación completa)
  ├─ docker-compose.yml
  ├─ part1-processes-threads/
  │   └─ task_processor.py
  │       └─ results_analysis.md
  ├─ part2-distributed-storage/
  │   └─ storage_system.py
  │       └─ mongo_setup.md
  ├─ part3-load-balancer/
  │   └─ load_balancer.py
  │       └─ load_test_results.md
  └─ docs/
      └─ final_report.pdf
```

En el README.md incluir:

- **Instrucciones** de ejecución paso a paso
- **Análisis** de resultados obtenidos
- **Conclusiones** sobre conceptos aplicados
- **Dificultades** encontradas y soluciones

Recomendaciones

- **Gestionar el tiempo:** Parte 1 (1 hora), Parte 2 (40 min), Parte 3 (20 min)
- **Probar incrementalmente:** Verificar cada componente antes de integrar
- **Documentar mientras desarrollas:** Incluir comentarios y observaciones
- **Usar los apuntes:** Consultar el material teórico proporcionado